

Optimisation Non-Linéaire 2024 - 2025

Enoncés d'implémentation

Question 1 : Backtracking vs. Wolfe.

Dans cette question, au lieu d'utiliser les conditions de Wolfe, on va utiliser une version simple de *backtracking* afin de calculer un pas α étant donné un itéré courant x et une direction de descente d . Cette procédure consiste à initialiser le pas à 1 et à diviser celui par 2 tant que l'objectif n'est pas inférieur à l'itéré courant. Une condition d'arrêt supplémentaire peut être implémentée lorsque le pas devient inférieur à une certaine tolérance (par exemple $1e-16$).

La fonction utilisée $f : \mathbb{R}^{m+1} \rightarrow \mathbb{R}$ est composée de $m+1$ variables $x_0, x_1, \dots, x_m$ et est définie par :

$$f(x) = \sum_{i:y^{(i)}=1} -\ln\left(\frac{1}{1 + e^{-(x_0+a_1^{(i)}x_1+a_2^{(i)}x_2+\dots+a_m^{(i)}x_m)}}\right) + \sum_{i:y^{(i)}=0} -\ln\left(1 - \frac{1}{1 + e^{-(x_0+a_1^{(i)}x_1+a_2^{(i)}x_2+\dots+a_m^{(i)}x_m)}}\right). \quad (1)$$

où les données $(a^{(i)}, y^{(i)})$ sont fournies dans des fichiers `.mat`.

Pour cela, on vous demande :

- Implémenter une fonction `def load_data(name):` où les paramètres d'entrée sont

- `name`, un nom de fichier,

et dont les paramètres de sortie sont

- `classe1`, l'ensemble des données $a^{(i)}$ de la classe 1 (celles où $y^{(i)} = 1$),
- `classe2`, l'ensemble des données $a^{(i)}$ de la classe 2 (celles où $y^{(i)} = 0$),

Cette fonction doit pouvoir traiter les fichiers `.mat` fournis.

- Implémenter une fonction

`def optimisation(classe1,classe2,methode="grad",pas="wolfe",maxiter=10**4,maxtime=10)`
où les paramètres d'entrée sont :

- `classe1`, l'ensemble des données d'entraînement $a^{(i)}$ de la classe 1,
- `classe2`, l'ensemble des données d'entraînement $a^{(i)}$ de la classe 2,
- `methode`, un paramètre permettant de choisir entre `grad` (méthode du gradient) et `gradacc` (méthode du gradient accéléré),
- `pas`, un paramètre permettant de choisir entre `wolfe` (pas calculé à l'aide des conditions de Wolfe) et `backtracking` (pas calculé à l'aide d'une simple procédure),

et dont les paramètres de sortie sont :

- `x`, les valeurs finales attribuées aux variables,
- `f`, l'ensemble des valeurs de la fonction objectif aux différents itérés calculés, et
- `t`, l'ensemble des instants auxquels la fonction objectif a été évaluée,

Dans cette fonction `optimisation`, plusieurs autres fonctions seront définies (voir ci-dessous), et, à l'aide celles-ci, les données seront utilisées. De plus, il ne faut pas oublier d'adapter les données au fait que le coefficient du premier terme des variables est une constante et vaut 1 pour toutes les données.

- Dans la fonction `optimisation`, implémenter une fonction `def fln(x)` où le paramètre d'entrée est :

- `x`, un vecteur de taille $m+1$,

et dont les paramètres de sortie sont :

- **f**, l'évaluation de la fonction f définie plus haut en x .
- **g**, le vecteur gradient en x de la fonction f définie plus haut.

Calculez d'abord les dérivées des fonctions suivantes :

$$f_1(x) = -\ln\left(\frac{1}{1+e^{-\alpha x}}\right) \quad \text{et} \quad f_2(x) = -\ln\left(1 - \frac{1}{1+e^{-\alpha x}}\right).$$

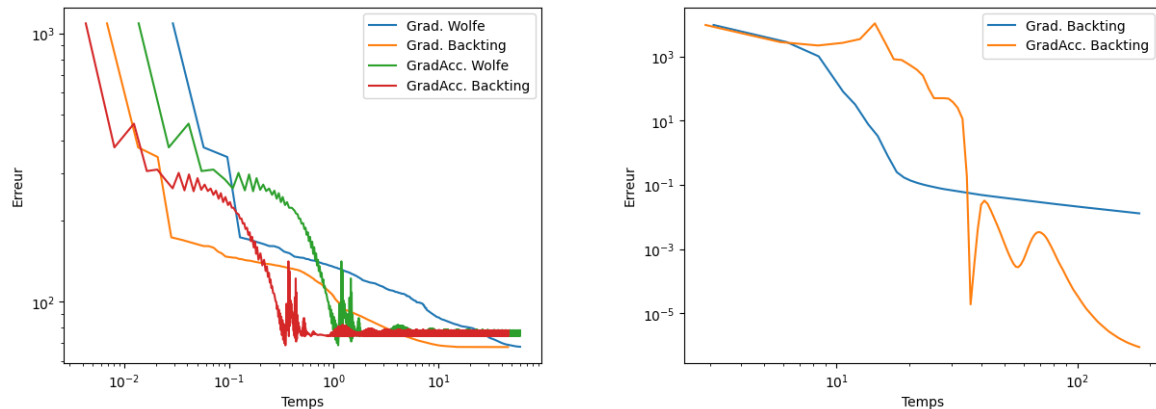
- **Dans** la fonction `optimisation`, implémenter les fonctions suivantes: Afin de pouvoir appliquer les conditions de Wolfe, on vous demande d'implémenter les fonctions suivantes :
 - `def w1(fct,x,d,alpha,beta1)`, retournant un booléen 1 si la condition W1 est vérifiée
 - `def w2(fct,x,d,alpha,beta2)`,retournant un booléen 1 si la condition W2 est vérifiée
 - `def wolfebisection(fct,x,d,alpha=1,beta1=0.0001,beta2=0.9)`, qui renvoie un pas α vérifiant les conditions de Wolfe
- **Dans** la fonction `optimisation`, implémenter une fonction `def backtrackingsimple(fct,x,d,alpha=1)` où les paramètres d'entrée sont :
 - `fct`, la référence à la fonction,
 - `x`, l'itéré courant,
 - `d`, la direction de descente en x ,
 - `alpha=1`, le pas initial,
 et dont le paramètre de sortie est :
 - `alpha`, le pas final.
- **Dans** la fonction `optimisation`, implémenter une fonction `def methgradient(fct,x0,maxiter=maxiter,maxtime=maxtime)` où les paramètres d'entrée sont :
 - `fct`, la référence à la fonction,
 - `x0`, l'itéré initial,
 - `maxiter=maxiter`, le nombre d'itéré maximal, à utiliser comme critère d'arrêt,
 - `maxtime=maxtime`, le temps limite, à utiliser comme critère d'arrêt,
 et dont les paramètres de sortie sont :
 - `x`, le dernier itéré calculé.
 - `fobj`, l'ensemble des valeurs de la fonction objectif pour les différents itérés calculés à l'aide de la méthode du gradient, et
 - `t`, l'ensemble des instants auxquels la fonction objectif a été évaluée.
- **Dans** la fonction `optimisation`, implémenter une fonction `def methgradientACC(fct,x0,maxiter=maxiter,maxtime=maxtime)` où les paramètres d'entrée sont :
 - `fct`, la référence à la fonction,
 - `x0`, l'itéré initial,
 - `maxiter=maxiter`, le nombre d'itéré maximal, à utiliser comme critère d'arrêt,

- `maxtime=maxtime`, le temps limite, à utiliser comme critère d'arrêt,

et dont les paramètres de sortie sont :

- `x`, le dernier itéré calculé.
- `fobj`, l'ensemble des valeurs de la fonction objectif pour les différents itérés calculés à l'aide de la méthode du gradient accéléré, et
- `t`, l'ensemble des instants auxquels la fonction objectif a été évaluée.

A l'aide de ces fonctions, on vous demande de générer deux figures :



La première, correspondant aux données de faibles dimensions :

- Comparaison des combinaisons gradient / gradient accéléré avec calcul du pas par Wolfe / backtracking
- `maxtime=60`

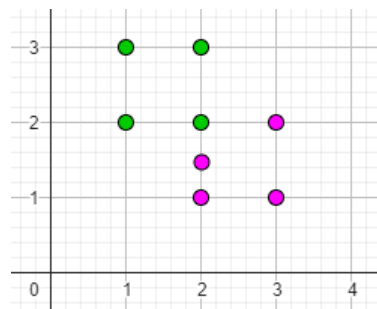
La deuxième, correspondant aux données de grandes dimensions :

- Comparaison entre gradient et gradient accéléré avec calcul du pas par backtracking
- `maxtime=180`

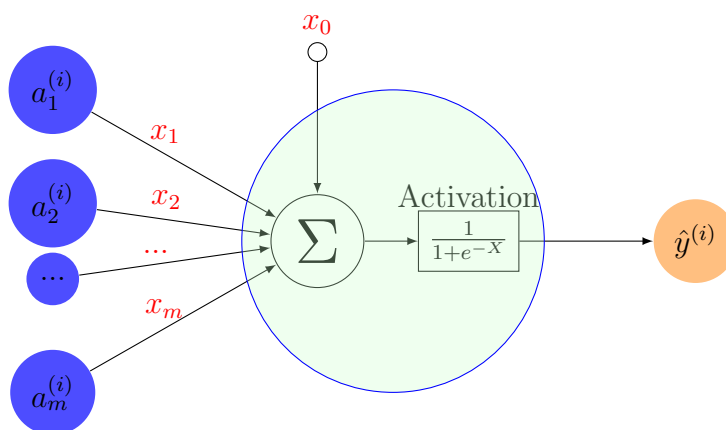
Question 2 : entraînement des poids d'un neurone.

Dans un ensemble d'entraînement, on possède n données notées $a^{(i)}$ dans la suite, avec $i = 1, \dots, n$. Chacune de ces données est un vecteur de dimension m . De plus, chacune de ces données appartient soit à la classe 1, soit à la classe 2. Ces informations sont indiquées par $y^{(i)} = 1$ si $a^{(i)}$ appartient à la classe 1, et $y^{(i)} = 0$ si $a^{(i)}$ appartient à la classe 2. Voici un petit exemple avec $n = 8$ données de dimension $m = 2$:

i	$a^{(i)}$	$y^{(i)}$
$i = 0$	$(1, 2)$	1
$i = 1$	$(1, 3)$	1
$i = 2$	$(2, 2)$	1
$i = 3$	$(2, 3)$	1
$i = 4$	$(2, 1.5)$	0
$i = 5$	$(2, 1)$	0
$i = 6$	$(3, 1)$	0
$i = 7$	$(3, 2)$	0



A l'aide de ces données, appelées données d'entraînement, on souhaite déterminer les poids $x_0, x_1, \dots, x_m$ d'un neurone (voir Figure ci-dessous) afin de permettre une séparation automatique de nouvelles données provenant d'un autre ensemble, appelé ensemble test.



La fonction d'activation du neurone utilisée ici est la sigmoïde (également appelée fonction logistique).

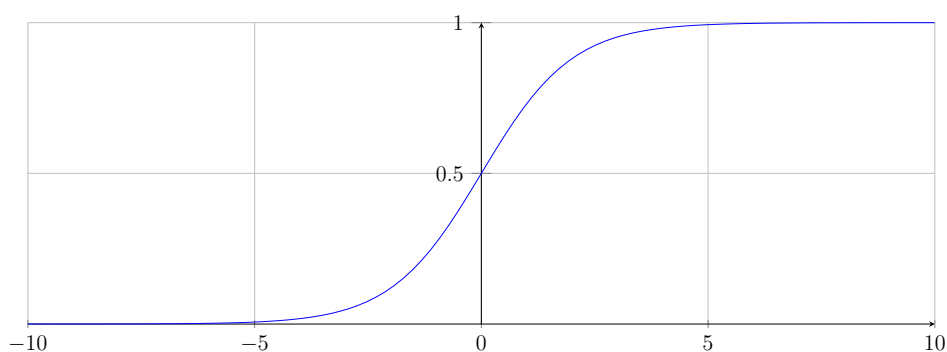


Figure 1: La fonction *sigmoïde* est la fonction $\frac{1}{1+e^{-X}}$ définie pour tout $X \in \mathbb{R}$.

Etant donné un vecteur d'entrée $a^{(i)}$ et les poids $x_0, x_1, \dots, x_m$, la sortie du neurone (la prédiction) est :

$$\hat{y}^{(i)} = \frac{1}{1 + e^{-(x_0 + a_1^{(i)} x_1 + a_2^{(i)} x_2 + \dots + a_m^{(i)} x_m)}}.$$

Dans l'idéal, il faudrait donc que les valeurs des poids $x_0, x_1, \dots, x_m$ soient telles que :

- lorsque l'entrée $a^{(i)}$ du neurone est un vecteur correspondant à la première classe, la sortie prédite $\hat{y}^{(i)}$ soit la plus proche possible de 1,
- lorsque l'entrée $a^{(i)}$ du neurone est un vecteur correspondant à la deuxième classe, la sortie prédite $\hat{y}^{(i)}$ soit la plus proche possible de 0.

Mais comment trouver ces valeurs idéales pour $x_0, x_1, \dots, x_m$ en utilisant les données d'entraînement ?

C'est là qu'intervient l'optimisation !

En utilisant l'ensemble d'entraînement, on va définir un problème d'optimisation où les variables seront les poids $x_0, x_1, \dots, x_m$ et la fonction objectif sera une mesure de l'écart entre les prédictions $\hat{y}^{(i)}$ et les valeurs attendues $y^{(i)}$. En notant cet écart $d(y^{(i)}, \hat{y}^{(i)})$, la fonction à minimiser sera

$$f(x) = \sum_{i=1}^n d(y^{(i)}, \hat{y}^{(i)}).$$

Il existe différents choix pour d afin de mesurer la distance entre $\hat{y}^{(i)}$ et $y^{(i)}$, entre-autres :

- $d(y^{(i)}, \hat{y}^{(i)}) = |y^{(i)} - \hat{y}^{(i)}|$,
- $d(y^{(i)}, \hat{y}^{(i)}) = (y^{(i)} - \hat{y}^{(i)})^2$, ou encore
- $d(y^{(i)}, \hat{y}^{(i)}) = -y^{(i)} \ln(\hat{y}^{(i)}) - (1 - y^{(i)}) \ln(1 - \hat{y}^{(i)})$.

Cette dernière fonction est appelée *l'entropie croisée* (cross-entropy loss), c'est une des mesures les plus utilisées dans le cadre des réseaux de neurones et c'est celle que nous utiliserons dans ce travail. Puisque $y^{(i)} \in \{0, 1\}$, remarquons que f peut se réécrire sous la forme suivante :

$$f(x) = \sum_{i=1}^n d(y^{(i)}, \hat{y}^{(i)}) = \sum_{i=1}^n -y^{(i)} \ln(\hat{y}^{(i)}) - (1 - y^{(i)}) \ln(1 - \hat{y}^{(i)}) = \sum_{i:y^{(i)}=1} -\ln(\hat{y}^{(i)}) + \sum_{i:y^{(i)}=0} -\ln(1 - \hat{y}^{(i)}).$$

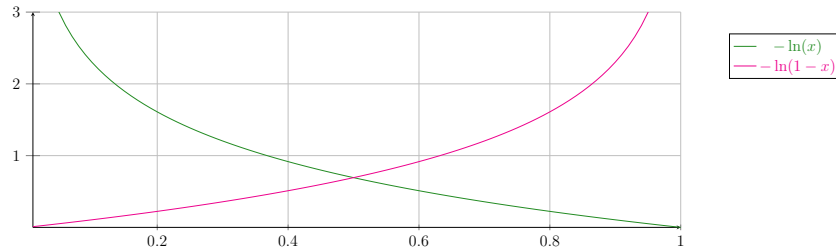


Figure 2: Les fonctions $-\ln(x)$ et $-\ln(1-x)$.

En résumé, afin que le neurone puisse effectuer son travail de classifieur binaire, les $m+1$ variables $x_0, x_1, \dots, x_m$ seront optimisées en minimisant la fonction d'erreur $f : \mathbb{R}^{m+1} \rightarrow \mathbb{R}$ définie par :

$$f(x) = \sum_{i:y^{(i)}=1} -\ln\left(\frac{1}{1 + e^{-(x_0 + a_1^{(i)} x_1 + a_2^{(i)} x_2 + \dots + a_m^{(i)} x_m)}}\right) + \sum_{i:y^{(i)}=0} -\ln\left(1 - \frac{1}{1 + e^{-(x_0 + a_1^{(i)} x_1 + a_2^{(i)} x_2 + \dots + a_m^{(i)} x_m)}}\right). \quad (2)$$

Après le processus d'optimisation, on espère que les poids identifiés $x_0, x_1, \dots, x_m$ permettront à la prédiction

$$\hat{y}^{(i)} = \frac{1}{1 + e^{-(x_0 + a_1^{(i)} x_1 + a_2^{(i)} x_2 + \dots + a_m^{(i)} x_m)}}$$

d'être proche de 1 si l'entrée $a^{(i)}$ appartient à la classe 1 et d'être proche de 0 si $a^{(i)}$ appartient à la classe 2, non seulement sur l'ensemble d'entraînement mais aussi sur un ensemble test !

Globalement, on souhaite que l'appel aux différentes fonctions soit simple et général :

```
1 ##### Entrainement du modele #####
2 classe1_train, classe2_train = load_data("train_set.mat")
3 poids_reseau = entraînement(classe1_train, classe2_train)
4
5 ##### Validation du modele #####
6 classe1_test, classe2_test = load_data("test_set.mat")
7 result = validation(poids_reseau, classe1_test, classe2_test)
```

Afin d'arriver à cela, on vous demande d'effectuer les tâches suivantes :

- Implémenter une fonction `def load_data(name):` où les paramètres d'entrée sont
 - `name`, un nom de fichier,

et dont les paramètres de sortie sont

- `classe1`, l'ensemble des données $a^{(i)}$ de la classe 1,
- `classe2`, l'ensemble des données $a^{(i)}$ de la classe 2,

Cette fonction doit pouvoir traiter les fichiers `.mat` fournis.

- Implémenter une fonction `def entraînement(classe1, classe2)` où les paramètres d'entrée sont :
 - `classe1`, l'ensemble des données d'entraînement $a^{(i)}$ de la classe 1,
 - `classe2`, l'ensemble des données d'entraînement $a^{(i)}$ de la classe 2,

et dont le paramètre de sortie est :

- `x`, les poids du neurone.

Dans cette fonction `entraînement`, plusieurs autres fonctions seront définies (voir ci-dessous), et, à l'aide celles-ci, les données seront utilisées et le problème d'optimisation décrit plus haut sera résolu afin d'identifier les poids optimaux. De plus, il ne faut pas oublier d'adapter les données au fait que le coefficient du premier terme du vecteur poids est une constante et vaut 1 pour toutes les données.

- **Dans** la fonction `entraînement`, implémenter une fonction `def fcrossentropy(x)` où le paramètre d'entrée est :
 - `x`, un vecteur de taille $m + 1$,

et dont les paramètres de sortie sont :

- `f`, l'évaluation de la fonction f définie plus haut en x .
- `g`, le vecteur gradient en x de la fonction f définie plus haut.

Calculez d'abord les dérivées des fonctions suivantes :

$$f_1(x) = -\ln\left(\frac{1}{1 + e^{-\alpha x}}\right) \quad \text{et} \quad f_2(x) = -\ln\left(1 - \frac{1}{1 + e^{-\alpha x}}\right).$$

- **Dans** la fonction `entraînement`, implémenter une fonction `def backtrackingsimple(fct, x, d, alpha=1)` où les paramètres d'entrée sont :
 - `fct`, la référence à la fonction,
 - `x`, l'itéré courant,
 - `d`, la direction de descente en x ,

- `alpha=1`, le pas initial,

et dont le paramètre de sortie est :

- `alpha`, le pas final.

Au lieu d'utiliser les conditions de Wolfe, on va utiliser une version simple de *backtracking* afin de calculer un pas α étant donné un itéré courant x et une direction de descente d . Cette procédure consiste à initialiser le pas à 1 et à diviser celui par 2 tant que l'objectif n'est pas inférieur à l'itéré courant. Une condition d'arrêt supplémentaire peut être implémentée lorsque le pas devient inférieur à une certaine tolérance (par exemple $1e-16$).

- Dans la fonction `entrainement`, implémenter une fonction `def methgradient(fct,x0,maxiter=10**3)` où les paramètres d'entrée sont :

- `fct`, la référence à la fonction,
- `x0`, l'itéré initial,
- `maxiter=10**3`, le nombre d'itéré maximal, à utiliser comme critère d'arrêt,

et dont les paramètres de sortie sont :

- `x`, le dernier itéré calculé.

A l'aide de ces fonctions, on vous demande de :

- Quel taux de prédiction pouvez-vous obtenir sur le jeu de données `test_set.mat` en entraînant votre modèle sur `train_set.mat` ?
 - Créer une figure qui montre l'évolution du taux de prédiction en fonction du pourcentage de données d'entraînement utilisées lors de l'entraînement (utiliser par exemple `np.random.sample(...)`).
 - Pour aller plus loin :
 - utiliser la méthode du gradient accéléré
 - utiliser un (beaucoup) plus grand jeu de données avec $n = 1398$ et $m = 43586$ est fourni par les fichiers `big_train_set.mat` et `big_test_set.mat` (données réelles venant de <http://kdd.ics.uci.edu/databases/20newsgroups/20newsgroups.html>).
(plusieurs éléments de votre code devront être adaptés car ces matrices sont sous forme *sparse*).
-

Quelques développements.

En reprenant l'exemple à $n = 8$ données de dimension $m = 2$ de la première page (qui sont les paramètres d'entrée *classe1* et *classe2* de la fonction **entraînement**), il est recommandé de les transformer sous la forme des matrices suivantes pour faciliter l'exécution des différentes fonctions définies à l'intérieur de **entraînement** :

$$a1 = \begin{pmatrix} 1 & 1 & 2 \\ 1 & 1 & 3 \\ 1 & 2 & 2 \\ 1 & 2 & 3 \end{pmatrix} \quad \text{et} \quad a2 = \begin{pmatrix} 1 & 2 & \frac{3}{2} \\ 1 & 2 & 1 \\ 1 & 3 & 1 \\ 1 & 3 & 2 \end{pmatrix}.$$

De plus, comme

$$f_1(x) = -\ln\left(\frac{1}{1+e^{-\alpha x}}\right) = \ln(1+e^{-\alpha x}), \quad \text{et}$$

$$f_2(x) = -\ln\left(1 - \frac{1}{1+e^{-\alpha x}}\right) = -\ln\left(\frac{e^{-\alpha x}}{1+e^{-\alpha x}}\right) = -\ln\left(\frac{1}{1+e^{\alpha x}}\right) = \ln(1+e^{\alpha x}),$$

la fonction (2) s'écrit :

$$f(x) = \sum_{i:b_1^{(i)}=1} \ln\left(1 + e^{-(x_0+a_1^{(i)}x_1+a_2^{(i)}x_2+\dots+a_m^{(i)}x_m)}\right) + \sum_{i:b_1^{(i)}=0} \ln\left(1 + e^{(x_0+a_1^{(i)}x_1+a_2^{(i)}x_2+\dots+a_m^{(i)}x_m)}\right).$$

Comme

$$f'_1(x) = (\ln(1+e^{-\alpha x}))' = -\alpha \frac{e^{-\alpha x}}{1+e^{-\alpha x}} = -\alpha \frac{1}{1+e^{\alpha x}}, \quad \text{et}$$

$$f'_2(x) = (\ln(1+e^{\alpha x}))' = \alpha \frac{e^{\alpha x}}{1+e^{\alpha x}} = \alpha \frac{1}{1+e^{-\alpha x}},$$

la dérivée partielle de f suivant la variable x_j est :

$$\frac{\partial f}{\partial x_j}(x) = \sum_{i:b_1^{(i)}=1} -a_j^{(i)} \frac{1}{1+e^{(x_0+a_1^{(i)}x_1+a_2^{(i)}x_2+\dots+a_m^{(i)}x_m)}} + \sum_{i:b_1^{(i)}=0} a_j^{(i)} \frac{1}{1+e^{-(x_0+a_1^{(i)}x_1+a_2^{(i)}x_2+\dots+a_m^{(i)}x_m)}}.$$

Le vecteur gradient peut donc s'écrire plus concisément :

$$\nabla f(x) = -a1^T v_1 + a2^T v_2,$$

où v_1 et v_2 sont respectivement des vecteurs de taille nb1 et nb2 :

$$v_1 = \begin{pmatrix} \frac{1}{1+e^{(x_0+a_1^{(1)}x_1+a_2^{(1)}x_2+\dots+a_m^{(1)}x_m)}} \\ \vdots \\ \frac{1}{1+e^{(x_0+a_1^{(nb1)}x_1+a_2^{(nb1)}x_2+\dots+a_m^{(nb1)}x_m)}} \end{pmatrix} \quad \text{et} \quad v_2 = \begin{pmatrix} \frac{1}{1+e^{-(x_0+a_1^{(1)}x_1+a_2^{(1)}x_2+\dots+a_m^{(1)}x_m)}} \\ \vdots \\ \frac{1}{1+e^{-(x_0+a_1^{(nb2)}x_1+a_2^{(nb2)}x_2+\dots+a_m^{(nb2)}x_m)}} \end{pmatrix}.$$